

Politechnika Świętokrzyska	
Wydział Elektrotechniki, Automatyki i Informatyki	
Przetwarzanie odporne na błędy	Temat projektu: 4 - CRC
Skład zespołu: • Przemysław Katuziński • Michał Kaczor • Jakub Kuśmierczyk • Bartłomiej Wydrzycki	Data wykonania sprawozdania: 05.11.2025r.

Link do repozytorium: https://github.com/Przemak19/POB_Project

Spis treści

1. Temat projektu	2
1.1. Główne założenia symulacji	2
1.2. Wykonane założenia projektowe.....	2
2. Opis i przedstawienie funkcjonalności symulacji	3
2.1. Interfejs użytkownika	3
2.1.1. Panel centralny (GraphPanel):.....	3
2.1.2. Panel sterowania (ControlPanel)	4
2.1.3. Panel logów (LogPanel).....	5
2.2. Obliczanie i weryfikacja CRC.....	6
2.2.1. Metody do obliczania i walidacji CRC	6
2.2.2. Główne metody pomocnicze	7
2.3. Symulacja.....	8
2.3.1. BlockingQueue – wysyłanie wiadomości.....	8
2.3.2. SimulationController	8
2.3.3. Node	9
2.3.4. Packet	10
2.3.5. NetworkGraph	10
2.3.6. Connection.....	11
2.4. Usterki.....	11
2.4.1. ErrorInjector	11
2.4.2. ErrorType	11
2.4.3. Fault.....	12
3. Podsumowanie projektu	12

1. Temat projektu

Symulacja pracy 10 komputerów połączonych w sieć o topologii grafu. Pomiędzy wybranymi przez użytkownika węzłami można przysyłać informacje zabezpieczone kodem CRC (dowolny wielomian wprowadzany przez użytkownika).

1.1. Główne założenia symulacji

Symulacja umożliwia:

- przesyłanie danych między komputerami (węzłami),
- symulację usterek i błędów sieciowych (takich jak utrata pakietów, opóźnienia, błędne bity, zawieszenie węzła, zmiana bitów),
- logowanie najważniejszych zdarzeń wpływających na symulację,
- wykrywanie błędów transmisji za pomocą kontroli CRC,
- automatyczną retransmisję pakietów po błędzie związanym z CRC na podstawie mechanizmu ACK/NACK,
- graficzne przedstawienie symulacji w prosty sposób.

1.2. Wykonane założenia projektowe

Wykonane zostały wszystkie założenia projektowe zaplanowane do wykonania w projekcie. Została dodana:

- implementacja kodowania i weryfikacji CRC,
- Szkielet zarządzania siecią
- Połączenie 10 węzłów o topologii grafu
- Komunikacja węzłów między sobą
- Stworzenie prostego okna aplikacji
- Panel z możliwością wysyłania wiadomości
- Prosta wizualizacja komputerów
- Wprowadzanie dowolnego wielomianu przez użytkownika
- Wyświetlanie podstawowych logów (monitorowanie zachowania systemu)
- Stworzenie funkcjonalności wprowadzenia 3 różnych błędów
- Stworzenie funkcjonalności usuwania usterek
- Rozwinięcie interfejsu użytkownika
- Zapewnienie stabilnego działania programu
- Stworzenie dokumentacji

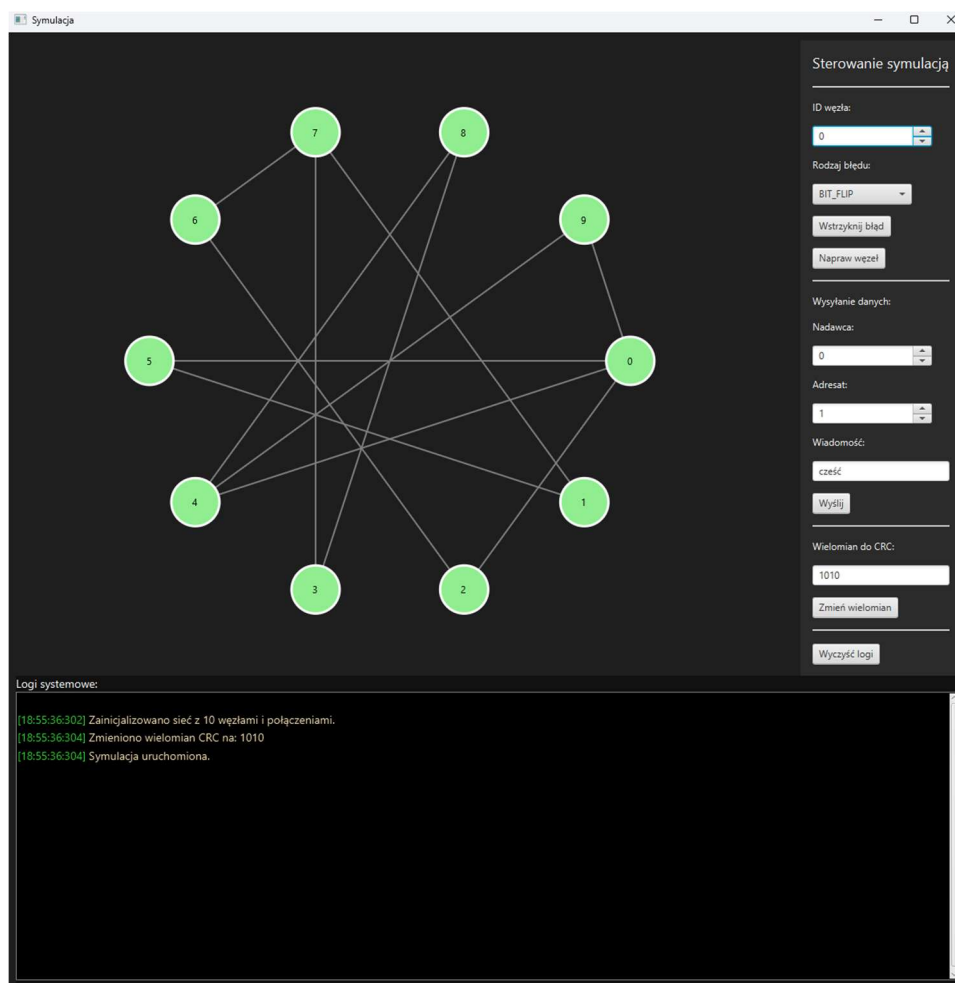
2. Opis i przedstawienie funkcjonalności symulacji

2.1. Interfejs użytkownika

Interfejs użytkownika aplikacji został zaprojektowany z wykorzystaniem technologii JavaFX, zapewniając intuicyjną i przejrzystą wizualizację działania sieci komputerowej.

Głównym celem interfejsu jest umożliwienie użytkownikowi obserwacji przebiegu symulacji, sterowania działaniem węzłów oraz analizy błędów i transmisji danych w czasie rzeczywistym.

Po uruchomieniu programu użytkownik widzi główne okno aplikacji zbudowane na bazie klasy MainWindow, która dzieli interfejs na trzy główne sekcje.

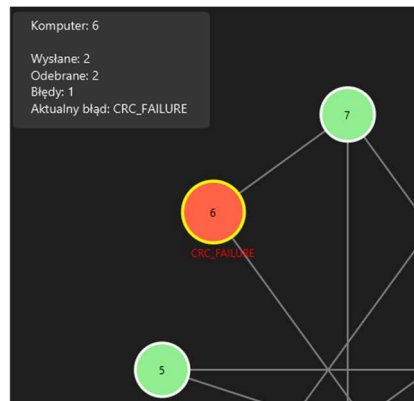


Rysunek 1 Interfejs użytkownika

2.1.1. Panel centralny (GraphPanel):

- W centrum okna znajduje się graficzna reprezentacja sieci komputerowej w postaci grafu.
- Każdy węzeł (Node) przedstawiony jest jako okrąg oznaczony numerem.
- Połączenia między węzłami (Connection) reprezentowane są jako linie.
- Kolory węzłów zmieniają się w zależności od ich stanu:
 - zielony – węzeł aktywny i gotowy do pracy,
 - czerwony – węzeł z błędem lub nieaktywny.

- Podczas działania symulacji widoczna jest animacja przepływu pakietów pomiędzy komputerami.
- Panel prezentuje również statystyki dla każdego węzła, takie jak liczba wysłanych, odebranych i błędnych pakietów (należy nakierować kursor na wybrany węzeł aby wyświetlić statystyki).

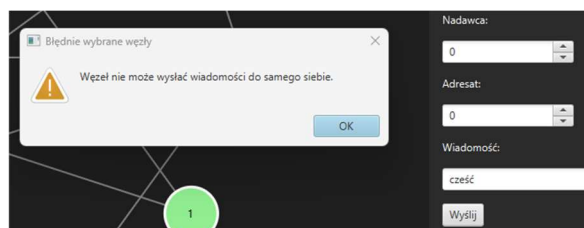


Rysunek 2 Statystyki węzła

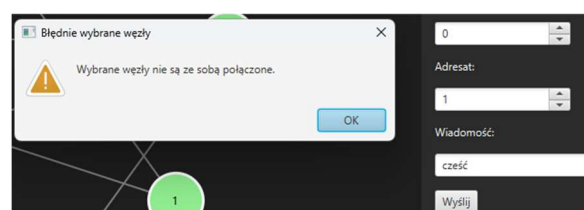
2.1.2. Panel sterowania (ControlPanel)

- Zawiera zestaw przycisków i pól umożliwiających interakcję z symulacją.
- Użytkownik może:
 - wybrać źródłowy i docelowy węzeł transmisji,
 - wprowadzić wiadomość do wysłania,
 - ustawić lub zmienić wielomian CRC,
 - rozpocząć transmisję pakietu,
 - wstrzyknąć wybrany błąd (ErrorType) do konkretnego węzła:
 - BIT_FLIP – zmiana bitu danych,
 - PACKET_DROP – utrata pakietu,
 - NODE_FREEZE – zatrzymanie węzła,
 - DELAY – opóźnienie transmisji,
 - CRC_FAILURE – awaria modułu CRC.
 - naprawić węzeł po wystąpieniu błędu.

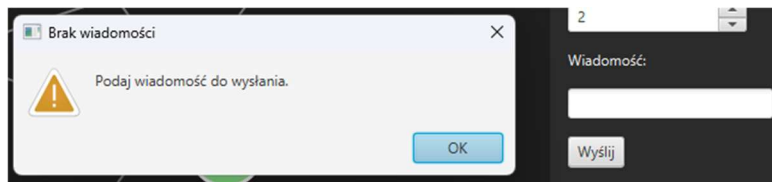
Zablokowane zostały sytuacje niezgodne z działaniem symulacji.



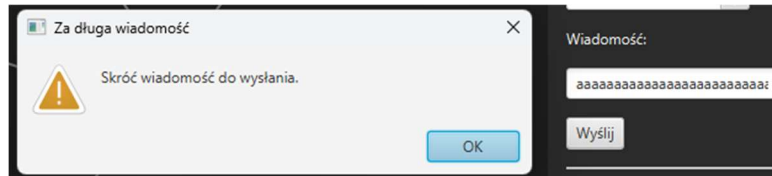
Rysunek 3 Brak możliwości wysłania wiadomości do tego samego węzła



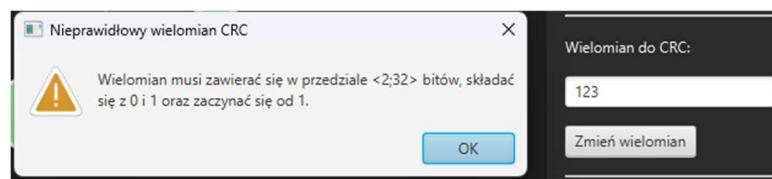
Rysunek 4 Brak możliwości wysłania wiadomości do niepołączonych węzłów



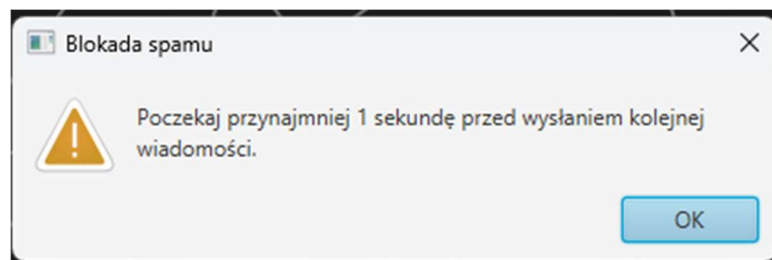
Rysunek 5 Brak możliwości wysłania pustych wiadomości



Rysunek 6 Wiadomość może mieć maksymalnie 200 znaków



Rysunek 7 Wielomian musi mieć określoną długość, składać się z 0 i 1



Rysunek 8 Blokada spamu

2.1.3. Panel logów (LogPanel)

- Wyświetla w czasie rzeczywistym działania i zdarzenia zachodzące w symulacji.
- Każdy wpis zawiera czas i opis zdarzenia.
- Log obejmuje:
 - rozpoczęcie i zakończenie transmisji,
 - wykrycie błędów CRC,
 - wysyłkę ACK/NACK,
 - utratę lub opóźnienie pakietu,
 - naprawę usterek.
- Dzięki temu użytkownik może dokładnie śledzić przebieg komunikacji i reakcję systemu na wstrzyknięte błędy.

```

Logi systemowe:
[18:55:36:302] Zainicjalizowano sieć z 10 węzłami i połączeniami.
[18:55:36:304] Zmieniono wielomian CRC na: 1010
[18:55:36:304] Symulacja uruchomiona.
[19:14:18:513] Komputer 6: wstrzyknięto błąd BIT_FLIP
[19:14:27:106] Komputer 6: wysłał pakiet do komputer 2 z wiadomością: cześć, CRC [bity]: 100
[19:14:27:309] Komputer 2: wykrył BŁĄD w pakiecie od komputer 6: cześć - CRC niepoprawne, CRC [bity]: 110
[19:14:27:511] Komputer 6: otrzymał NACK - retransmisja pakietu.
[19:14:27:511] Komputer 6: usterka BIT_FLIP usunięta.
[19:14:27:511] Komputer 6: wysłał pakiet do komputer 2 z wiadomością: cześć, CRC [bity]: 100
[19:14:27:712] Komputer 2: odebrał POPRAWNY pakiet od komputer 6 cześć, CRC [bity]: 100
[19:14:27:913] Komputer 6: otrzymał potwierdzenie ACK od komputer 2.
[19:14:40:189] Komputer 6: wstrzyknięto błąd CRC_FAILURE
[19:20:25:134] Komputer 6: usterka CRC_FAILURE usunięta.
[19:36:00:332] Komputer 3: wstrzyknięto błąd CRC_FAILURE
[19:36:09:655] Zmieniono wielomian CRC na: 111010101
[19:36:28:459] Komputer 3: błąd przy obliczaniu CRC.

```

Rysunek 9 Logi systemowe kolorujące najważniejsze informacje

2.2. Obliczanie i weryfikacja CRC

Obliczenia związane z CRC znajdują się w klasie CRCUtil, jest to klasa obliczająca kod CRC metodą dzielenia modulo-2 (XOR). Działa dla dowolnego wielomianu binarnego wprowadzonego przez użytkownika.

2.2.1. Metody do obliczania i walidacji CRC

Metoda do obliczania CRC:

```

/**
 * Oblicza CRC dla dowolnego tekstu.
 * @param message Tekst do zabezpieczenia.
 * @param polynomial Dowolny wielomian.
 * @return CRC w postaci binarnej.
 */
public String computeCRC(String message, String polynomial) {
    //jeśli wiadomość jest pusta zwracamy pustą CRC
    if (message == null || message.isEmpty()) return "";

    //M(x) - zmiana wiadomości w ciąg bitów
    String bits = toBinary(message);
    //k - długość wielomianu G(x)
    int polyLen = polynomial.length();

    //M'(x) - dopisujemy do danych k-1 zer, gdzie k to długość wielomianu
    StringBuilder padded = new StringBuilder(bits);
    for (int i = 0; i < polyLen - 1; i++) padded.append('0');

    //M'(x)
    char[] data = padded.toString().toCharArray();
    //G(x)
    char[] poly = polynomial.toCharArray();

    //Dzielimy M'(x) przez G(x) modulo 2
    //Działanie modulo 2 oznacza, że zamiast odejmowania robimy XOR
    //Jeśli pierwszy bit dzielnej jest 1, wykonujemy XOR z wielomianem
    //Jeśli jest 0, przesuwamy się dalej
    for (int i = 0; i <= data.length - polyLen; i++) {
        if (data[i] == '1') {
            for (int j = 0; j < polyLen; j++) {
                data[i + j] = xorBit(data[i + j], poly[j]);
            }
        }
    }

    //CRC - pozostałe bity
    StringBuilder crc = new StringBuilder();
    for (int i = data.length - (polyLen - 1); i < data.length; i++) {
        crc.append(data[i]);
    }
}

```

```

    }

    //Zwróć CRC
    return crc.toString();
}

```

Metoda do walidacji CRC:

```

/**
 * Sprawdza poprawność CRC, jeśli wynik składa się z zer, zwraca true.
 * @param packet Pakiet z danymi.
 * @param polynomial Wielomian, przy pomocy którego dane są zabezpieczone.
 * @return Wartość {@code true} lub {@code false}.
 */
public boolean validateCRC(Packet packet, String polynomial) {

    int polyLen = polynomial.length();
    char[] data = packet.getData().toCharArray();
    char[] poly = polynomial.toCharArray();

    // Dzielenie modulo 2 (XOR)
    for (int i = 0; i <= data.length - polyLen; i++) {
        if (data[i] == '1') {
            for (int j = 0; j < polyLen; j++) {
                data[i + j] = xorBit(data[i + j], poly[j]);
            }
        }
    }

    // Sprawdź, czy reszta = same zera
    for (int i = data.length - (polyLen - 1); i < data.length; i++) {
        if (data[i] != '0') {
            return false; // CRC błędne
        }
    }

    return true; // CRC poprawne
}

```

2.2.2. Główne metody pomocnicze

Główne metody pomocnicze do obliczania i walidacji:

```

/**
 * Pomocnicza metoda, która używa StandardCharsets.UTF_8.
 * Tworzy prawdziwe bajty, a nie pojedyncze znaki, aby korzystać z polskich znaków.
 * @param message Wiadomość do przekształcenia.
 * @return Wiadomość w postaci binarnej.
 */
private String toBinary(String message) {
    byte[] bytes = message.getBytes(StandardCharsets.UTF_8);
    StringBuilder bits = new StringBuilder();
    for (byte b : bytes) {
        bits.append(String.format("%8s", Integer.toBinaryString(b & 0xFF)).replace(' ',
'0'));
    }
    return bits.toString();
}

/**
 * Wykonuje xor na bitach.
 * @param a Pierwszy bit.
 * @param b Drugi bit.
 * @return Wynik działania xor.
 */
private char xorBit(char a, char b) {
    return (a == b) ? '0' : '1';
}

/**
 * Łączy wiadomość z wygenerowanym CRC.
 * @param message Wiadomość do połączenia z CRC.
 * @param polynomial Wielomian do zabezpieczenia wiadomości.
 * @return Ciąg bitów z CRC.
 */
public String appendCRC(String message, String polynomial) {
    return toBinary(message) + computeCRC(message, polynomial);
}

```

2.3. Symulacja

Symulacja opiera się na współpracujących klasach Node, Connection, NetworkGraph, Packet, SimulationController.

2.3.1. BlockingQueue – wysyłanie wiadomości

W symulacji sieci każdy węzeł (Node) działa jako niezależny wątek, który potrafi wysyłać i odbierać pakiety danych (Packet). Aby zachować realizm działania sieci i umożliwić równoległe operacje, zastosowano mechanizm kolejek współdzielonych, każda instancja Node posiada własną kolejkę przychodzących pakietów.

BlockingQueue<Packet> to wielowątkowa kolejka służąca do bezpiecznego przekazywania pakietów pomiędzy węzłami. Każdy węzeł posiada własną kolejkę incomingQueue, do której inne węzły wstawiają pakiety podczas wysyłki. Metody put() i take() z klasy BlockingQueue zapewniają synchronizację, jeśli węzeł nie ma żadnych pakietów do odebrania, czeka aż jakiś się pojawi (blokada wątku).

2.3.2. SimulationController

- Zarządza całym przebiegiem symulacji.
- Tworzy sieć 10 węzłów (Node) i łączy je.
- Umożliwia zmianę wielomianu CRC (sprawdza poprawność wprowadzonego wzorca).
- Steruje startem symulacji, wstrzykiwaniem błędów oraz naprawą węzłów.
- Przechowuje odniesienie do warstwy graficznej (GraphPanel), dzięki której można wizualizować sieć.

```

/**
 * Inicjalizuje sieć poprzez utworzenie 10 węzłów i zdefiniowanie podstawowych połączeń
 * między nimi.
 */
public void initializeNetwork() {
    // Tworzenie 10 węzłów
    for (int i = 0; i < 10; i++) {
        Node node = new Node(i, this);
        graph.addNode(node);
    }

    // Tworzenie prostych połączeń (graf liniowy)
    for (int i = 0; i < 5; i++) {
        graph.addConnection(graph.getNodes().get(i), graph.getNodes().get(i + 4));
    }
    graph.addConnection(graph.getNodes().get(0), graph.getNodes().get(2));
    graph.addConnection(graph.getNodes().get(0), graph.getNodes().get(5));
    graph.addConnection(graph.getNodes().get(1), graph.getNodes().get(7));
    graph.addConnection(graph.getNodes().get(9), graph.getNodes().get(0));
    graph.addConnection(graph.getNodes().get(8), graph.getNodes().get(3));
    graph.addConnection(graph.getNodes().get(4), graph.getNodes().get(9));
    graph.addConnection(graph.getNodes().get(6), graph.getNodes().get(7));

    Logger.log("Zainicjalizowano sieć z 10 węzłami i połączeniami.");
}

/**
 * Uruchamia symulację, aktywując wszystkie węzły w sieci oraz ustawiając domyślny wielomian
 * CRC.
 */
public void startSimulation() {
    for (Node node : graph.getNodes()) {
        node.start();
    }
    setCrcPolynomial("1010");
    Logger.log("Symulacja uruchomiona.");
}

```

2.3.3. Node

- Reprezentuje komputer w sieci, który może wysyłać i odbierać pakiety.
- Działa w osobnym wątku (implementuje Runnable), dzięki czemu możliwa jest równoległa komunikacja.
- Dla każdej wysyłki:
 - oblicza sumę CRC i dołącza ją do wiadomości,
 - symuluje błędy transmisji (np. zmiana bitów, opóźnienie, utrata),
 - wysyła pakiet (Packet) do innego węzła.
- Odbierając pakiet:
 - weryfikuje poprawność CRC,
 - w razie błędu wysyła negatywne potwierdzenie (NACK) i żąda retransmisji,
 - w razie poprawności wysyła ACK.
- Zbiera statystyki (liczba wysłanych, odebranych i błędnych pakietów).

```

/**
 * Główna pętla wątku węzła.
 * Oczekuje na pakiety w kolejce i przetwarza je sekwencyjnie.
 */
@Override
public void run() {
    while (true) {
        try {
            Packet packet = incomingQueue.take();
            receivePacket(packet);
        }
    }
}

```

```

    } catch (InterruptedException e) {
        Logger.log("Węzeł " + id + " został zatrzymany.");
        break;
    }
}
}

```

2.3.4. Packet

- Reprezentuje jednostkę danych przesyłaną między węzłami.
- Zawiera:
 - treść wiadomości (w formacie bitów),
 - identyfikatory nadawcy i odbiorcy,
 - informacje o opóźnieniu,
 - flagi informujące, czy to ACK i czy jest pozytywny/negatywny.

```

/**
 * Tworzy nowy pakiet z danymi przesyłanymi między dwoma węzłami.
 * @param data Zawartość pakietu (np. dane lub wiadomość).
 * @param sourceId Identyfikator węzła nadawcy.
 * @param destinationId Identyfikator węzła odbiorcy.
 */
public Packet(String data, int sourceId, int destinationId) {
    this.data = data;
    this.sourceId = sourceId;
    this.destinationId = destinationId;
    this.delay = 200;
}

/**
 * Tworzy specjalny pakiet typu ACK (potwierdzenie lub NACK).
 * @param sourceId Identyfikator węzła wysyłającego potwierdzenie.
 * @param destinationId Identyfikator odbiorcy potwierdzenia.
 * @param ackPositive Wartość {@code true} - ACK pozytywny, {@code false} - żądanie
 * retransmisji.
 * @return Nowy pakiet potwierdzający.
 */
public static Packet createAckPacket(int sourceId, int destinationId, boolean ackPositive) {
    Packet ack = new Packet("ACK", sourceId, destinationId);
    ack.isAck = true;
    ack.ackPositive = ackPositive;
    return ack;
}

```

2.3.5. NetworkGraph

NetworkGraph jest centralnym elementem zarządzającym strukturą sieci.

Jest używany przez klasę SimulationController do:

- inicjalizacji sieci i tworzenia węzłów,
- dodawania połączeń między nimi (budowania topologii),
- udostępniania danych o sieci pozostałym komponentom GUI.

```

/** Lista wszystkich węzłów w grafie. */
private final List<Node> nodes = new ArrayList<>();

/** Lista wszystkich połączeń (krawędzi) pomiędzy węzłami. */
private final List<Connection> connections = new ArrayList<>();

```

2.3.6. Connection

Klasa Connection reprezentuje połączenie pomiędzy dwoma węzłami sieci w symulacji. Jest to podstawowy element struktury grafu sieciowego, umożliwia odwzorowanie relacji i komunikacji pomiędzy komputerami wirtualnej sieci.

Każde połączenie składa się z dwóch końców:

- nodeA - pierwszy węzeł połączenia,
- nodeB - drugi węzeł połączenia.

```
/**
 * Tworzy połączenie pomiędzy dwoma węzłami sieci.
 * @param a Pierwszy węzeł połączenia.
 * @param b Drugi węzeł połączenia.
 */
public Connection(Node a, Node b) {
    this.nodeA = a;
    this.nodeB = b;
}
```

2.4. Usterki

Usterki (symulacja błędów) w programie są reprezentowane przez obiekty klasy Fault w węzłach, które są określone za pomocą typu wyliczeniowego ErrorType i wstrzykiwane przy pomocy ErrorInjector.

2.4.1. ErrorInjector

Klasa ErrorInjector odpowiada za symulację błędów w sieci komputerowej. Jej głównym zadaniem jest wstrzykiwanie określonego rodzaju usterki do wybranego węzła (Node), co pozwala badać zachowanie systemu w warunkach awarii. W symulacji odgrywa rolę narzędzia testowego, umożliwia celowe generowanie błędów takich jak utrata pakietu, błędne CRC czy zamrożenie węzła.

Klasa ta jest używana przez SimulationController do celowego wprowadzania błędów w symulacji. Umożliwia testowanie odporności protokołu komunikacyjnego oraz skuteczności mechanizmów wykrywania i naprawy błędów (ACK/NACK, retransmisja).

```
/**
 * Wstrzykuje wybrany typ błędu do podanego węzła.
 * @param node Węzeł, do którego ma zostać wprowadzony błąd.
 * @param type Typ błędu, który ma zostać wstrzyknięty.
 */
public void applyError(Node node, ErrorType type) {
    node.injectFault(type);
}
```

2.4.2. ErrorType

ErrorType to typ wyliczeniowy definiujący wszystkie możliwe rodzaje błędów, jakie można wstrzyknąć do węzłów sieci. Każda wartość odpowiada innemu scenariuszowi awarii, który wpływa na zachowanie węzła lub transmisji danych.

Rodzaje błędów:

- BIT_FLIP - Zmienia losowy bit w danych pakietu, powodując uszkodzenie informacji.
- PACKET_DROP - Powoduje utratę pakietu, nie trafia on do odbiorcy.
- NODE_FREEZE - „Zamraża” węzeł, przestaje on odbierać i wysyłać dane.
- DELAY - Wprowadza losowe opóźnienie w transmisji pakietu.

- CRC_FAILURE - Generuje błąd w module CRC.

2.4.3. *Fault*

Klasa Fault reprezentuje pojedynczą usterkę (błąd) przypisaną do konkretnego węzła sieci (Node). Przechowuje informację o typie błędu (ErrorType) oraz o jego aktualnym stanie (czy jest aktywny).

Działanie:

- Tworząc nowy obiekt Fault, przekazuje się typ błędu, np. ErrorType.PACKET_DROP.
- Błąd jest automatycznie oznaczany jako aktywny.
- Po zakończeniu symulacji błędu (np. po naprawie przez użytkownika lub retransmisji) metoda deactivate() dezaktywuje usterkę.

```
/**
 * Tworzy nową usterkę danego typu i automatycznie ją aktywuje.
 * @param type Typ błędu, który ma zostać wstrzyknięty.
 */
public Fault(ErrorType type) {
    this.type = type;
    this.active = true;
}
```

3. Podsumowanie projektu

Projekt przedstawia kompletną symulację działania sieci komputerowej z mechanizmem obsługi błędów transmisji danych przy użyciu kodów CRC. System został zaprojektowany w języku Java z wykorzystaniem wielowątkowości oraz interfejsu graficznego opartego o JavaFX.

Projekt w pełni realizuje założenia symulacji i komunikacji z obsługą błędów i kodów CRC.

Osiągnięte cele obejmują:

- zrozumienie i implementację mechanizmu CRC,
- zaprojektowanie i wizualizację sieci o dynamicznym zachowaniu,
- obsługę różnych rodzajów błędów transmisji,
- zastosowanie programowania wielowątkowego i asynchronicznej komunikacji,
- stworzenie intuicyjnego interfejsu graficznego do obserwacji i kontroli działania systemu.